

激光器串口控制系统 — 项目说明书

一、项目概述

本项目是一个基于 **Qt 5 (C++)** 开发的**三路激光器串口控制系统**，通过串口（UART）与下位机（激光器硬件设备）通信，实现对三台激光器的远程状态监控与电流调节。

- **开发框架**：Qt 5，使用 **QSerialPort** 模块进行串口通信
- **编程语言**：C++
- **项目名称**：serial（生成的可执行文件为 **serial.exe**）
- **界面尺寸**：1100 × 750 像素
- **通信参数**：波特率 115200，数据位 8，停止位 1，无校验，无流控

二、文件结构及各文件作用

文件	类型	作用
<code>serialhelper_new.pro</code>	Qt 项目文件	定义项目配置：引用的 Qt 模块、源文件列表、目标文件名、图标
<code>main.cpp</code>	C++ 源文件	程序入口，初始化应用、设置编码、显示主窗口
<code>widget.h</code>	C++ 头文件	声明 Widget 类的所有成员变量和成员函数
<code>widget.cpp</code>	C++ 源文件	Widget 类的全部功能实现（核心代码，约 630 行）
<code>widget.ui</code>	Qt UI 文件	XML 格式的界面布局定义（用 Qt Designer 设计）
<code>serialhelp.ico</code>	图标文件	应用程序的图标

三、各文件代码详细讲解

3.1 `serialhelper_new.pro` — 项目配置文件

```
QT += core gui serialport      # 引入 Qt 核心、GUI、串口三个模块
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets # Qt5+ 需要单独引入 widgets
TARGET = serial                # 生成的可执行文件名为 serial(.exe)
TEMPLATE = app                 # 项目类型为应用程序
SOURCES += main.cpp widget.cpp # 源文件列表
HEADERS += widget.h           # 头文件列表
FORMS += widget.ui            # UI 界面文件
RC_ICONS = serialhelp.ico     # Windows 下的程序图标
```

关键点：`serialport` 模块是 Qt 提供的串口通信 API，本项目所有的串口读写功能都依赖此模块。

3.2 `main.cpp` — 程序入口

```
QTextCodec::setCodecForLocale(QTextCodec::codecForName("UTF-8"));
```

作用：设置全局默认编码为 UTF-8，确保串口接收的中文数据能正确显示，防止出现乱码问题。

`Widget w; w.show();` 创建主窗口并显示，然后进入 Qt 事件循环 `a.exec()`。

3.3 widget.h — 类声明

声明了 `Widget` 类，继承自 `QWidget`。主要包括以下几类成员：

a) 公开方法（public）

方法	作用
<code>sendLaserCommand(int laserIndex, char cmd)</code>	向指定激光器发送控制指令（单字符）
<code>setLaserReady(int laserIndex, bool ready)</code>	设置激光器的就绪/未就绪状态并更新 UI

b) 私有槽函数（private slots） — 响应 UI 事件

槽函数	触发时机
<code>on_openBt_clicked()</code>	点击"打开串口"按钮
<code>on_closeBt_clicked()</code>	点击"关闭串口"按钮
<code>on_sendBt_clicked()</code>	点击"发送数据"按钮
<code>on_clearBt_clicked()</code>	点击"清空文本"按钮
<code>serialPortReadyRead_Slot()</code>	串口收到数据时自动调用
<code>on_laser1/2/3spinbox_valueChanged()</code>	激光器电流调节框数值变化
<code>checkSerialPorts()</code>	定时器触发：检查串口插拔状态
<code>autoReconnectSerialPort()</code>	定时器触发：自动重连串口
<code>handleSerialError()</code>	串口发生错误时调用
<code>checkLaserStatus()</code>	定时器触发：定期验证激光器状态

c) 关键成员变量

状态变量：

- `currentLaser1/2/3mA`：三个激光器当前显示电流值（mA）
- `previousSpinValue1/2/3`：上一次的显示值，用于判断用户是调大还是调小

就绪状态（两层）：

- `laser1/2/3RawReady`：从串口接收到的原始就绪状态（下位机直接上报）

- `laser1/2/3Ready`：经过依赖逻辑计算后的最终就绪状态

定时器：

定时器	周期	作用
<code>serialCheckTimer</code>	1 秒	检测串口拔插
<code>autoReconnectTimer</code>	2 秒	断线后自动重连
<code>statusCheckTimer</code>	3 秒	检查激光器依赖关系

防抖控制：

- `lastSentTimers[3]`：每个激光器独立的时间戳记录，最小发送间隔 120ms，防止按钮重复触发导致指令风暴

3.4 `widget.cpp` — 核心功能实现

这是整个项目最核心的文件，约 630 行代码，实现全部业务逻辑。

3.4.1 构造函数 `Widget::Widget()`（第 7~70 行）

初始化流程：

1. 创建串口对象 `serialPort`
2. 初始化三个定时器并启动
3. 连接定时器信号到对应的槽函数
4. 连接串口错误信号 `errorOccurred` 到 `handleSerialError`
5. 填充可用串口列表 → `refreshSerialPortList()`
6. 设置界面字体（微软雅黑，支持中文）
7. 设置三个激光器的电流范围：
 - Laser1：0 ~ 1000 mA
 - Laser2：0 ~ 1000 mA
 - Laser3：800 ~ 10000 mA
8. 初始化模式下拉框（粗调/细调/默认）
9. 连接串口数据就绪信号 `readyRead`

3.4.2 串口打开 — `on_openBt_clicked()`（第 305~345 行）

用户选择串口号 → 点击"打开串口"

↓

设置端口名、波特率(115200)、数据位(8)、停止位(1)、校验(无)、流控(无)

↓

`open(QIODevice::ReadWrite)` 打开串口

↓

记录端口名、启用自动重连、清空接收缓冲、重置激光器状态

3.4.3 串口关闭 — `on_closeBt_clicked()` (第 348~357 行)

关闭串口、禁用自动重连、重置所有激光器状态。

3.4.4 数据接收 — `serialPortReadyRead_Slot()` (第 360~475 行)

这是最复杂的数据处理函数，完整流程：

```
串口有数据到达 (readyRead 信号触发)
↓
读取全部数据 → 追加到 rxBuffer (接收缓冲区)
↓
按换行符 '\n' 分割，逐行处理
↓
UTF-8 解码 (失败则回退到 GBK 编码)
↓
特殊行合并逻辑：以 ':' 开头或非 Laser 开头的行，与上一行合并显示
↓
状态检测：匹配多种格式的状态关键词
- "Laser1:温度就绪" / "Laser1就绪" / "L1:OK" 等 → 标记为就绪
- "Laser1:温度未就绪" / "Laser1未就绪" / "L1:NG" 等 → 标记为未就绪
↓
状态有变化时 → 调用 updateAllLaserStates() 更新依赖链
```

编码兼容性设计 (`decodeSerialData`, 第 294~302 行)：先用 UTF-8 解码，如果出现替换字符 `U+FFFD`，则回退到 GBK 编码。这种双编码策略确保了下位机无论发送 UTF-8 还是 GBK 数据都能正确解析。

3.4.5 激光器控制 — `sendLaserCommand()` (第 478~512 行)

发送前的安全检查链：

1. 串口是否打开？
2. 激光器索引是否有效 (1~3) ？
3. 该激光器是否满足依赖条件 (`canControlLaser`) ？
4. 距上次发送是否超过 120ms (防抖) ？

全部通过后，写入单字节指令到串口。

3.4.6 激光器电流调节逻辑

Laser1 和 Laser2 (相同逻辑)：

```
spinbox 值变化 → 判断方向 (比上一次大=调大)
↓
读取模式：粗调 (步进 10mA) / 细调 (步进 1mA)
↓
更新 currentLaserXmA 值
```

↓

根据方向和模式选择指令字符发送

激光器	粗调↑	粗调↓	细调↑	细调↓
Laser1	'1'	'0'	'3'	'2'
Laser2	'4'	'5'	'6'	'7'

Laser3（固定步进 100mA）：

方向	指令
↑	'8'
↓	'9'

关键设计：spinbox 仅作为"方向指示器"使用。用户上滚/下滚 spinbox 时，实际改变的是内部维护的 `currentLaserXmA` 变量值，然后用 `blockSignals` 回写 spinbox 显示。这意味着 **spinbox** 显示的是程序内部状态，而非用户直接输入，巧妙避免了用户乱输入的问题。

3.4.7 激光器依赖关系（核心业务逻辑）

三个激光器存在链式依赖：

Laser1 就绪（独立，仅依赖自己的原始状态）
↓
Laser2 就绪 = Laser1 就绪 && Laser2 原始状态就绪
↓
Laser3 就绪 = Laser1 就绪 && Laser2 就绪 && Laser3 原始状态就绪

含义：Laser2 需要 Laser1 先就绪才能工作，Laser3 需要 Laser1 和 Laser2 都就绪才能工作。这是一种硬件安全连锁机制。

```
// canControlLaser() - 检查是否能控制指定激光器
laserIndex=1 → 仅需 laser1Ready
laserIndex=2 → 需 laser1Ready && laser2Ready
laserIndex=3 → 需 laser1Ready && laser2Ready && laser3Ready
```

3.4.8 热插拔检测 — `checkSerialPorts()`（第 202~243 行）

每 1 秒执行一次：

- 1. 获取当前系统所有可用串口列表
- 2. 与上一次记录的列表对比
- 3. 如果有变化：
 - 刷新串口下拉列表

- 如果之前打开的串口现在不存在了 → 自动关闭并提示断开
 - 如果之前打开的串口重新出现 → 启动 2 秒延迟的自动重连
4. 实时更新"打开串口"按钮的颜色和文字（绿色=已连接）

3.4.9 自动重连 — `autoReconnectSerialPort()`（第 246~282 行）

断线后自动恢复连接：

1. 检查目标端口是否存在
2. 以相同参数（115200/8/N/1）重新打开串口
3. 成功后清空缓冲区、重置激光器状态
4. 失败则继续每 2 秒重试

3.4.10 UI 状态更新 — `setLaserReady()`（第 515~540 行）

根据就绪状态更新界面：

- **就绪**：状态标签显示绿色"就绪"，spinbox 启用
- **未就绪**：状态标签显示红色"未就绪"，spinbox 禁用（无法操作）

3.5 `widget.ui` — UI 布局（第 1~591 行）

界面分为左、右两个区域：

左侧（主操作区）

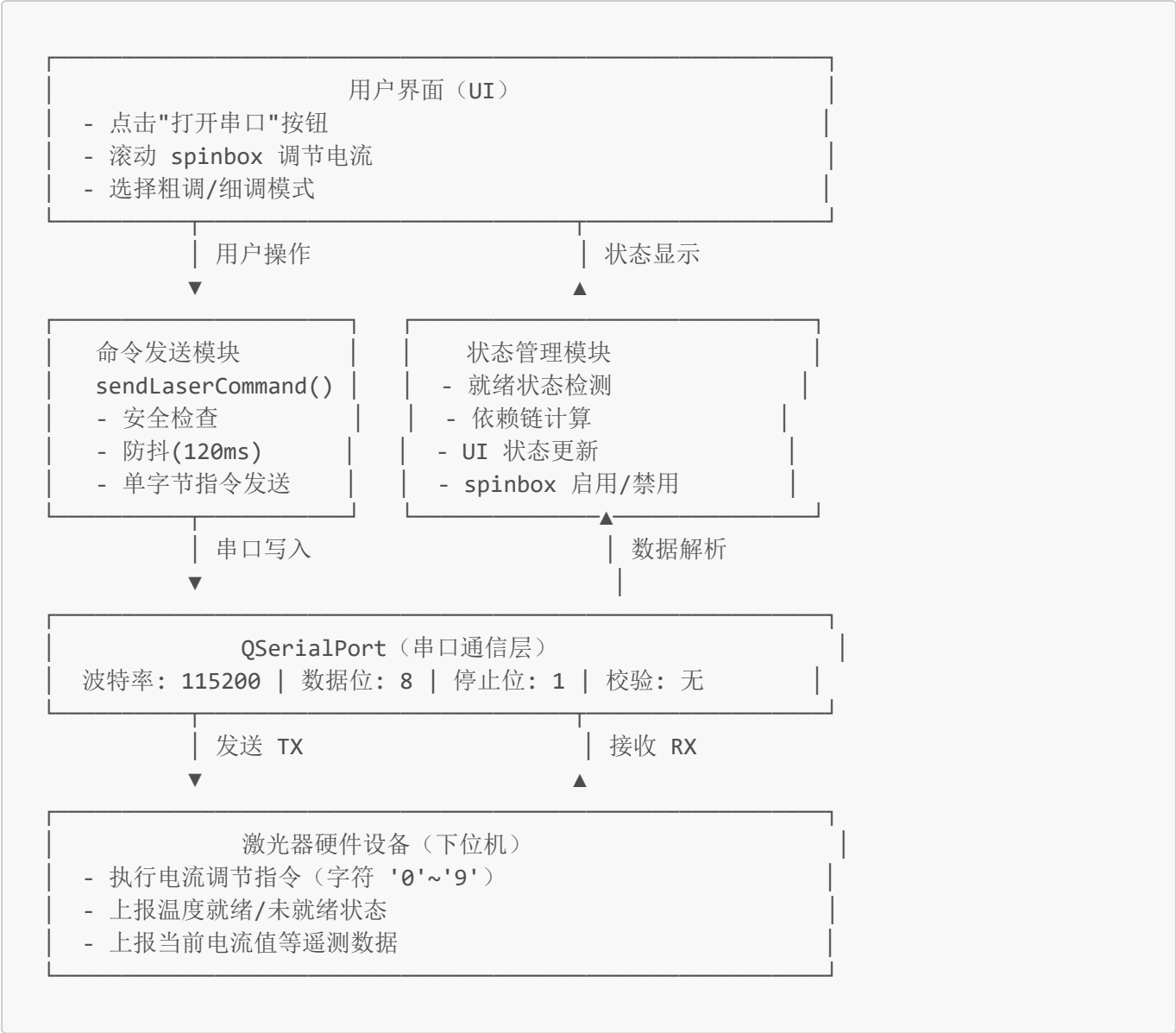
区域	组件	说明
接收区	<code>QPlainTextEdit</code> (receiveEdit)	宽 791 × 高 491，只读，显示串口接收数据和系统日志
串口配置	<code>QComboBox</code> × 5	串口号、波特率(4800/9600/115200)、数据位(5/6/7/8)、停止位(1/1.5/2)、校验位(NULL)
发送区	<code>QLineEdit</code> (sendEdit)	手动输入要发送的文本
按钮区	<code>QPushButton</code> × 4	打开串口、关闭串口、发送数据、清空文本

右侧（激光器控制区）

三个 `QGroupBox`，自上而下排列，每个包含：

显示项	组件	说明
状态指示	<code>QLabel</code> (laserXStatusLabel)	显示"就绪"（绿）或"未就绪"（红）
模式选择	<code>QComboBox</code> (laserXModeCb)	Laser1/2：粗调/细调；Laser3：默认
输入电流	<code>QSpinBox</code> (laserXspinbox)	数值调节框 + "mA" 标签

四、程序运行原理（整体数据流）



五、背景运行机制（三个定时器）

定时器	周期	功能
<code>serialCheckTimer</code>	每 1 秒	扫描系统串口列表变化 → 检测热插拔 → 触发自动重连
<code>autoReconnectTimer</code>	每 2 秒	断线后持续尝试重新打开目标串口
<code>statusCheckTimer</code>	每 3 秒	验证激光器依赖关系是否仍然满足

这三个定时器在后台持续运行，无需用户手动干预，实现了**全自动化的串口连接管理**。

六、功能清单

功能	说明
串口连接管理	手动选择串口、打开/关闭，支持 115200/8/N/1 参数配置

功能	说明
三路激光器电流控制	通过 spinbox 滚轮调节，Laser1/2 支持粗调($\pm 10\text{mA}$)和细调($\pm 1\text{mA}$)，Laser3 固定步进($\pm 100\text{mA}$)
激光器就绪监控	实时解析下位机上报的状态信息，自动识别就绪/未就绪
链式安全连锁	Laser2 依赖 Laser1，Laser3 依赖 Laser1+2，未满足依赖时禁止操作
热插拔检测	1 秒轮询检测 USB 串口设备的插入和拔出
自动重连	设备拔出后自动关闭，重新插入后 2 秒内自动恢复连接
双编码兼容	同时支持 UTF-8 和 GBK 编码的串口数据，防止中文乱码
指令防抖	每个激光器 120ms 内不重复发送，避免指令风暴
状态日志	所有串口事件（连接、断开、接收、发送、错误）都显示在接收区

七、指令协议（与下位机通信）

程序通过串口发送**单字符 ASCII 指令**控制激光器：

指令字符	激光器	含义
'0'	Laser1	电流减小（粗调， -10mA ）
'1'	Laser1	电流增大（粗调， $+10\text{mA}$ ）
'2'	Laser1	电流减小（细调， -1mA ）
'3'	Laser1	电流增大（细调， $+1\text{mA}$ ）
'4'	Laser2	电流增大（粗调， $+10\text{mA}$ ）
'5'	Laser2	电流减小（粗调， -10mA ）
'6'	Laser2	电流增大（细调， $+1\text{mA}$ ）
'7'	Laser2	电流减小（细调， -1mA ）
'8'	Laser3	电流增大（ $+100\text{mA}$ ）
'9'	Laser3	电流减小（ -100mA ）

下位机上报的状态格式支持多种匹配模式（容错设计），包括：

- Laser1:温度就绪 / Laser1:温度未就绪
- Laser1就绪 / Laser1未就绪
- Laser1Ready / Laser1NotReady（大小写不敏感）
- L1:OK / L1:NG

八、关键技术要点

1. **spinbox 作为方向指示器**：spinbox 不直接表示真实电流值，而是通过比较本次值与上次值（`previousSpinValue`）判断用户意图（调大/调小），实际电流值由内部变量 `currentLaserXmA` 维护。这解决了 spinbox 原生行为与步进式控制不匹配的问题。
2. **防重复发送**：`QElapsedTimer` 记录每个激光器上次发送时间，120ms 内的重复请求被直接丢弃，避免用户快速滚动 spinbox 时产生指令风暴。
3. **状态双层设计**：区分"原始状态"（`RawReady`，来自下位机）和"最终状态"（`Ready`，经依赖逻辑计算），确保链式安全锁的正确性。
4. **热插拔鲁棒性**：通过定时轮询 `QSerialPortInfo::availablePorts()` 实现，不依赖 Windows 消息（WM_DEVICECHANGE），跨平台兼容性更好。
5. **串行错误自动恢复**：`QSerialPort::ResourceError` 错误（通常是设备被拔出）触发自动重连流程，无需用户手动干预。

九、构建与运行

编译环境：Qt 5.x + MinGW 或 MSVC 编译器

构建步骤：

```
qmake serialhelper_new.pro
make          # Linux / MinGW
nmake        # MSVC
```

运行：启动 `serial.exe`，选择串口号 → 点击"打开串口" → 等待激光器状态变为绿色"就绪" → 滚动 spinbox 调节电流。

十、总结

本项目是一个完整的**工业级激光器串口控制系统**，具备以下亮点：

- **安全性**：三层依赖连锁 + 发送前状态校验，防止未就绪时误操作
- **可靠性**：热插拔检测 + 自动重连 + 串口错误恢复，保证长时间稳定运行
- **易用性**：直观的 GUI 界面 + 实时状态反馈 + 中英文双编码兼容
- **工程性**：防抖设计 + 缓冲区拼接 + 编码回退，考虑了各种实际使用场景中的边界情况